

YaP Vehicles — real vehicle mechanics

Not minecarts or boats. YaP Vehicles gives plugin authors a **buildable chassis** plus **non-vanilla physics** (traction, slip, slopes, yaw inertia, fuel, damage).

Runs on the product path: **YaP-Folia** (`folia-supported: true`). Build with `./scripts/build-yap-folia.sh`.

Piece	Where
Runtime + API	<code>plugins/yap-vehicles.jar</code> (Folia-aware plugin YaP Vehicles)
Operator packaging	<code>modules/yap-vehicles-module.jar</code> (<code>provides: [vehicles]</code>)
Example author plugin	<code>examples/yap-vehicle-addon/</code>

Operator install

GAMEPLAY opt-in — not in the CORE default release:

```
gradle installGameplayDefaults
# or: gradle assembleRelease -PyapGameplay=true
# or:
gradle :vehicles-plugin:installIntoPlugins
gradle :vehicles-module:installIntoModules
YAP_INCLUDE_VEHICLES=1 gradle prepareClientPack
./scripts/start.sh --fg
```

Artifact	Path
Plugin	<code>plugins/yap-vehicles.jar</code>
Module	<code>modules/yap-vehicles-module.jar</code>
Client pack	<code>resourcepacks/yapcore-default.zip</code> (Faithful + Vehicles when gameplay pack built)
- Config: <code>plugins/YaP Vehicles/config.yml</code>	
- Pack: <code>resource-pack-file=yapcore-default.zip</code> in <code>config/server.properties</code>	
- Commands: <code>/yapvehicle</code> <code>spawn list destroy types give adapt shop upgrades reload</code>	

Artifact	Path
- Controls: WASD drive · jump = brake · sprint = boost · sneak = handbrake	
- Builtins: <code>chassis</code> , <code>buggy</code> , <code>hoverbike</code> , plus fleet	
(<code>truck_4x4</code> , <code>monster_truck</code> , <code>sport_car</code> , <code>hypercar</code> , <code>lambo</code> , <code>ferrari</code> ,	
<code>mclaren</code> , <code>porsche</code>) when <code>builtins.fleet: true</code>	
- Spawn items are branded YaP Chassis tokens (paper + PDC) — not minecarts	
- Fuel: coal by default — sneak + right-click vehicle with coal to refuel	
- Upgrades: craft / shop / sneak-install parts (tires, sizes, lift kits, ...)	
- Glass + interior: clear glass windshields/side windows; seats, dash, wheel inside cabin	
- Shop: <code>/yapvehicle shop</code> — vehicles (top) + parts (bottom)	
- High-res models: <code>ItemDisplay</code> bodies (CMD 77200+) when <code>visuals.high-res-models: true</code>	
— pack: <code>resourcepacks/yapcore-default.zip</code> (includes YaP Vehicles)	
- Web dashboard: Vehicles tab at <code>http://127.0.0.1:8080/</code> — WEB_DASHBOARD.md	

Chassis (what authors build on)

Do **not** hijack minecart/boat entities or textures. Start from a kit:

Kit	Use
<code>ChassisKit.bare()</code>	Rectangular skeleton + wheel hubs + mounts
<code>ChassisKit.car()</code>	Cabin + clear glass + interior + 3 seats
<code>ChassisKit.sport()</code> / <code>hyper()</code>	Low sports / hyper frames
<code>ChassisKit.fourByFour()</code>	Tall 4x4 with bed seat
<code>ChassisKit.monster()</code>	Huge wheels + high cabin
<code>ChassisKit.bike()</code>	Narrow frame
<code>ChassisKit.truck()</code>	Long frame + bed

Named mounts: `hood` , `cabin` , `roof` , `bed` , `bumper_f` / `bumper_r` , `wheel_fl` / `fr` / `rl` / `rr` , `driver` .

```
ChassisBlueprint frame = ChassisKit.car();
api.registerType(VehicleType.builder("my_car")
    .displayName("My Car")
    .chassis(frame) // YaP frame geometry
    .bodyPanel(frame, "hood", Material.BLUE_CONCRETE, 1.2, 0.35, 0.7)
    .bodyPanel(frame, "cabin", Material.BLUE_CONCRETE, 1.1, 0.5, 0.9)
    .traction(1.0).lateralGrip(0.28).yawInertia(0.35) // non-vanilla physics
    .fuel(1000, 0.4)
    .health(40)
    .build());
```

Invisible ArmorStand = authority root; **BlockDisplay** frame = what players see. Spawn token material defaults to `ChassisKit.SPAWN_TOKEN` (`PAPER`) — minecart/boat materials are rejected by the builder.

Physics (not in vanilla)

System	What it does
Engine curve	Accel falls off toward max speed
Surface grip	Ice / sand / mud / paved change traction + rolling resistance
Lateral slip	Yaw while moving injects sideways drift; grip kills it
Yaw inertia	Steering is a rate with mass feel, not instant snap
Slope force	Grade pushes downhill / resists uphill
Handbrake	Sneak cuts lateral grip for drifts
Rolling resistance	Separate from air drag
Fuel / damage	Throttle burns fuel; collisions (and hits) damage HP

Read live state: `vehicle.getSpeed()` , `vehicle.getLateralSpeed()` .

Author quick start

1. Soft-depend on YaPVehicles in `plugin.yml` :

```
softdepend: [YaPVehicles]
```

2. Compile against `vehicles-plugin` .

3. Register via `ServicesManager` (see chassis example above).

Full example: [examples/yap-vehicle-addon](#).

Fuel (coal)

Config under `fuel:` in `plugins/YaPVehicles/config.yml`:

Key	Default	Meaning
<code>item</code>	<code>COAL</code>	Material consumed to refuel
<code>per-item</code>	<code>200</code>	Fuel units added per item
<code>burn-multiplier</code>	<code>1.0</code>	Global burn scale (stacks with upgrades)
<code>require-sneak</code>	<code>true</code>	Sneak + RMB vehicle with fuel item

Per-type burn still comes from `VehicleType.fuel(max, perTick)`. Effective burn = `perTick × burn-multiplier × upgrade fuelBurnMul`.

Upgrades (parts)

One part per **slot** (`ENGINE`, `TIRES`, `ARMOR`, `TANK`, `UTILITY`). Installing another part in the same slot replaces the old one. Installed parts persist on the chassis via PDC.

Defaults

Id	Slot	Effect
<code>turbo_engine</code> / <code>eco_carburetor</code>	ENGINE	Speed/accel vs efficiency
<code>tires_street</code> / <code>sport_tires</code> / <code>tires_offroad</code> / <code>tires_mud</code> / <code>tires_slick</code>	TIRES	Compound grip / slope / speed
<code>wheels_plus1</code> / <code>plus2</code> / <code>deep_dish</code> / <code>monster</code>	WHEELS	Visual tire scale + ride
<code>lift_2in</code> / <code>4in</code> / <code>6in</code> / <code>lift_monster</code>	SUSPENSION	Ride height + slope tradeoffs
<code>reinforced_armor</code>	ARMOR	+HP
<code>fuel_tank_xl</code>	TANK	+fuel capacity
<code>racing_suspension</code> / <code>nitro_kit</code>	UTILITY	Handling / boost

Lift kits change **ride height** in physics; wheel sizes **rescale WHEEL visuals** and nudge clearance. Tire compounds change traction / lateral / slope grip.

- **Craft:** shapeless recipes (when `upgrades.craft-enabled`)
- **Shop:** `/yapvehicle shop` — vehicles + parts for materials
- **Install:** sneak + right-click vehicle while holding the part
- **Give:** `/yapvehicle upgrades give <id>`

Fleet vehicles (shop + spawn)

Id	Class
<code>truck_4x4</code>	4×4 truck
<code>monster_truck</code>	Monster truck
<code>sport_car</code>	Sport
<code>hypercar</code>	Hyper
<code>lambo</code> / <code>ferrari</code> / <code>mclaren</code> / <code>porsche</code>	Exotics

Glass uses clear `GLASS` BlockDisplays (see-through). Interiors include seats, dash, steering wheel, and console visible through the cabin.

High-res models

With `visuals.high-res-models: true` (default), each fleet type attaches an **ItemDisplay** body using CustomModelData **77200-77210** from [resourcepacks/yap-vehicles](#) . Textures are **64×64**; JSON models live under `assets/yapvehicles/models/vehicle/` .

```
builder.bodyModel(HighResModels.LAMBO, 0, 0.5, 0, 2.2);
```

Clients must load `yapcore-default.zip` (or a pack that merges YaP Vehicles). Without the pack, HD bodies look like blank paper — set `visuals.high-res-models: false` for BlockDisplay-only bodies.

Preview gallery: [resourcepacks/yap-vehicles/showcase/](#) — demo PNGs of every model (regenerate with `python3 scripts/generate-vehicle-showcase.py`).

Author API

```
VehicleUpgradeAPI ups = api.upgrades();
ups.register(VehicleUpgrade.builder("my_supercharger")
    .displayName("Supercharger")
    .slot(UpgradeSlot.ENGINE)
    .icon(Material.PAPER, 77200) // add CMD override in your pack
    .stats(StatModifier.builder().maxSpeedMul(1.4).fuelBurnMul(1.3).build())
    .craft(Material.DIAMOND, 4)
    .craft(Material.COAL, 16)
    .shopPrice(Material.DIAMOND, 6)
```

```
.shopSlot(19)
.build());
ups.install(vehicle, ups.get("my_supercharger").orElseThrow(), player);
```

Custom item icons

Upgrade items are `paper` + **CustomModelData** `77101–77107`.

Ship pack: merged into [resourcepacks/yapcore-default.zip](#) (also standalone [yap-vehicles.zip](#)). Default server config uses `resource-pack-file=yapcore-default.zip`. Replace PNGs under `assets/yapvehicles/textures/item/` with final art; authors should use CMD $\geq$ **77200** and add paper overrides.

API surface

Type	Role
<code>ChassisKit</code> / <code>ChassisBlueprint</code>	Buildable frames + mount points
<code>VehicleAPI</code>	Register types, spawn/destroy, lookup
<code>VehicleUpgradeAPI</code> / <code>VehicleUpgrade</code>	Parts registry, craft/shop items, install
<code>StatModifier</code> / <code>UpgradeSlot</code>	Stat buffs + mount slots
<code>VehicleType</code>	Seats, visuals, physics knobs, fuel, health
<code>Vehicle</code>	Live instance — enter/exit, fuel, damage, slip
<code>VehicleController</code>	Input → throttle/steer/brake/boost/handbrake
<code>VehicleVisual</code>	Extra <code>ItemDisplay</code> / <code>BlockDisplay</code> body panels
Events	spawn, destroy, enter, exit, collide, fuel-empty, damage

```
Bukkit.getServicesManager().load(VehicleAPI.class);
```

Threading

All spawn / enter / physics run on Bukkit **SYNC**. Do not call `VehicleAPI` async.

Compat layer (other vehicle plugins)

Many vehicle plugins spawn **minecarts/boats** and hang custom models / `ModelEngine` / resource packs on them. YaP can **claim** those entities and drive them with our chassis physics while optionally **keeping the foreign entity as a synced visual**.

Setting (config.yml → compat)	Meaning
enabled	Master switch
require-marker	Only claim if custom name / tag / foreign PDC present
ignore-vanilla	Leave plain unnamed carts/boats alone
preserve-model	Glue foreign entity to chassis; hide YaP frame displays
default-type	YaP type when no map matches (usually chassis)
name-map / tag-map	Route specific names/tags → YaP types
known-plugins	Soft list of vehicle plugin names

Triggers: mounting or right-clicking a claimable minecart/boat, or `/yapvehicle adapt [type]` while looking at one.

API for authors / host hooks:

```
VehicleAPI api = Bukkit.getServicesManager().load(VehicleAPI.class);
api.compat().registerHook(foreign -> {
    // return Optional.of("buggy"), Optional.of("skip"), or empty
    return Optional.empty();
});
api.compat().adapt(minecartEntity, player, null);
```

Register `VehicleCompatHook` from a small bridge plugin if you need per-plugin PDC / Mythic / ItemsAdder detection beyond the generic heuristics.

vs vanilla minecarts / boats

	Minecart / boat	YaP Vehicles
Entity	Vanilla vehicle	Invisible chassis + BlockDisplay frame
Item	Minecart / boat	Branded YaP Chassis token
Physics	Rails / water	Traction, slip, slopes, inertia
Extensibility	Limited	Kits + mounts + controllers
Other plugins' models	Native	Compat layer remaps onto YaP physics

YaP Vehicles does **not** replace rails. Use minecarts for tracks; use this API for cars, bikes, and custom vehicles. Use **compat** when another plugin already spawned a minecart/boat “car”.

Build

```
gradle :vehicles-plugin:installIntoPlugins  
gradle :vehicles-module:installIntoModules  
gradle :yap-vehicle-addon:jar
```